

One Person Lab

OPL App 白皮书

让复杂知识工作拥有一个本地优先、云端连续、结果可复查的专业工作台

OPL App 的核心承诺，是把 OPL Framework 的阶段式交付能力、系列专业智能体和本地/云端资源，变成用户每天能打开、能理解、能继续、能复查的专业工作台。

OPL App / Docker WebUI / OPL Workspace / Foundry Agents

2026-07-04

Public whitepaper

目录

1	定位摘要	2
2	为什么不是再做一个聊天框	2
3	OPL App 的答案：可信专业工作台	2
4	一个工作台，不用跳工具	3
5	专业智能体：用户先看到工作目的	3
6	结果带来路	3
7	工作台跟着工作走	4
8	用已有资源，不重建世界	4
9	为什么用户可以相信 OPL App 专业	4
10	与 Framework、Cloud 和 Foundry Agents 的关系	5
11	用户会如何感知 OPL App	5
12	本文边界	5
13	结语	6

发布日期：2026-07-04

适用对象：希望理解 OPL App 为什么存在、怎样承接 OPL 系列智能体、如何从本机自然走向云端工作空间，以及为什么它能作为专业知识工作入口的用户、合作者、早期采用者和技术决策者。

核心判断：OPL App 的核心承诺，是把 OPL Framework 的阶段式交付能力、系列专业智能体和本地/云端资源，变成用户每天能打开、能理解、能继续、能复查的专业工作台。

1 定位摘要

- OPL App 是 One Person Lab 面向用户的工作台，不是又一个普通聊天框，也不是通用 agent dashboard。
- OPL App 的第一层价值，是让用户从一个入口进入科研、基金、演示、书稿、智能体构建和通用复杂知识工作。
- OPL App 的第二层价值，是让长期任务有可见进度、产物、卡点、审阅、回执和下一步，而不是只留下对话记录或最终文件。
- OPL App 的第三层价值，是让同一套工作台在本机、浏览器和云端 Workspace 中连续使用：macOS 桌面 App、Docker/WebUI、OPL Workspace 共享同一套任务、产物、进度和回执语言。
- OPL App 消费 OPL Framework、OPL Cloud 和 Foundry Agents 的能力，但不替它们重新定义运行真相、资源归属或领域质量判断。
- OPL App 的专业感来自克制：用户先看到工作目的和可继续的成果，内部的 agent、runtime、shell、provider、contract 和 release evidence 留在该出现的位置。

2 为什么不是再做一个聊天框

AI 已经能回答问题、写一段文字、生成一段代码，也能在单次对话里帮用户改一份材料。问题出现在工作变长以后。

一篇论文不是一次问答。一份基金申请不是一段生成文本。一套正式汇报不是几张临时图片。它们会经历材料整理、方向判断、初稿生成、审阅、修订、重画图、重写段落、补证据、改格式、再审查、再交付。用户离开几天后回来，需要知道工作做到哪里，哪个文件是当前版本，哪些判断已经形成，哪里卡住，下一步该让谁继续。

普通聊天产品把注意力放在“这轮回答得怎么样”。复杂知识工作更需要回答：

- 我的项目从哪里继续？
- 这个结果是怎么来的？
- 哪些材料、代码、环境、审阅和回执支持它？
- 当前是 AI 继续做、我来决定，还是某个专业智能体需要重新审阅？
- 如果今天在本机开始，明天想在服务器或云端继续，是否还在同一条工作线上？

OPL App 处理的就是这类问题。它把对话、文件、运行进度、专业智能体、产物和回执放进一个可继续的工作台。用户不需要在聊天、终端、文件夹、网页、云资源面板和人工记录之间来回拼接。

3 OPL App 的答案：可信专业工作台

OPL App 可以理解为 One Person Lab 的日常工作面。用户打开 App，不是先研究底层系统，而是直接进入工作：选择工作目录，描述要做的任务，选择工作目的，查看当前进度，打开产物，处理需要自己决定的事项。

这个工作台遵循三个表达：

一个工作台，不用跳工具。研究、基金、演示、书稿和智能体构建都从同一入口进入。用户按工作目的选择，而不是按后端、仓库、命令、provider 或 shell 选择。

结果带来路。OPL App 不只展示最终文件。它让任务、材料、产物、审阅、回执、卡点和下一步连在一起，让用户知道结果从哪里来，为什么可以继续，什么时候需要回到专业审阅。

工作台跟着工作走。用户可以从本机桌面 App 开始，也可以通过 Docker/WebUI 在 Linux、Windows、服务器或云主机上用浏览器打开同一套工作台。当 WebUI 被云端托管，并绑定账号、存储、隔离和资源策略时，它就是 OPL Workspace。

这三个表达比“设置中心”“可替换界面”“合同定义真相”更接近用户感受到的价值。内部工程仍然重要，但白皮书要解释的是用户为什么相信这个产品能承接正式工作。

4 一个工作台，不用跳工具

复杂知识工作通常分散在很多地方：聊天窗口里有想法，文件夹里有材料，终端里有脚本，浏览器里有文献和数据源，云端有计算资源，人工记录里有下一步。分散本身会消耗用户注意力，也会削弱结果可信度。

OPL App 的第一件事，是把入口统一起来。用户面对的不是一排技术后端，而是清楚的工作目的：

- 科研：围绕研究问题、数据、文献、分析、证据包、稿件和投稿材料工作。
- 基金：围绕选题、创新点、技术路线、申请书、模拟评审和修订工作。
- 演示：围绕汇报故事线、PPT、图表、视觉交付和导出工作。
- 书稿：围绕大纲、章节、风格、修订和出版交接工作。
- 智能体构建：围绕新 agent 的蓝图、能力、测试、运行边界和改进工作。

这些入口背后可以分别对应 MAS、MAG、RCA、OPL BookForge、OPL Meta Agent 等 Foundry Agents。普通用户不需要先记住缩写。用户先选择自己要完成什么成果，OPL App 再把任务路由到对应专业能力和工作线。

这也是 OPL App 与通用 agent dashboard 的区别。通用 dashboard 常常把 agent、model、tool、permission、backend、provider 都暴露给用户。OPL App 先把这些收敛成工作目的，让用户从“我要做什么”开始，而不是从“我要选哪个后端”开始。

5 专业智能体：用户先看到工作目的

OPL 系列智能体像一个专业团队。不同智能体负责不同成果，但它们共享同一套 App 工作台语言。

Med Auto Science 面向科研和医学研究工作。用户看到的是科研任务：文献、数据、研究问题、分析、证据包、稿件和投稿材料。背后需要 MAS 保留研究事实、发表质量检查、产物权威和负责人确认。

Med Auto Grant 面向基金工作。用户看到的是基金方向、申请书结构、创新点、技术路线、预算和评审说服力。背后需要 MAG 保留基金事实、可资助性判断、模拟评审和修订边界。

RedCube AI 面向演示和视觉交付。用户看到的是汇报、PPT、图表、故事线、视觉风格和导出包。背后需要 RCA 保留视觉交付质量、渲染、审阅和导出边界。

OPL BookForge 面向书稿和长文档。用户看到的是大纲、章节、风格、修订、图表和出版交接。背后需要 BookForge 保留书稿结构、素材组织和出版交接边界。

OPL Meta Agent 面向智能体构建和机制改进。用户看到的是“我要做一个能稳定完成某类工作的 agent”。背后需要 OMA 把目标、能力、测试、运行边界和可接力改进组织起来。

OPL App 的职责不是替这些智能体做专业裁决，而是让用户能在同一个工作台里进入它们、观察它们、接收它们的产物和回执，并在需要时继续下一轮。

6 结果带来路

正式知识工作的结果不能只是一份文件。用户需要知道结果是怎样形成的。

一张图应该能回到数据、代码、参数、环境和审阅记录。一段论文结论应该能回到材料、分析、证据和 reviewer 意见。一页 PPT 应该能回到故事线、图表来源、修改记录和导出状态。一个智能体能力包应该能回到目标、测试、边界、运行记录和改进原因。

OPL App 把这些信息表达成用户能理解的工作线：

任务目标 -> 材料和资源 -> 执行过程 -> 产物 -> 审阅/检查 -> 回执 -> 下一步

用户回来时，不应该只看到“任务完成”或“任务失败”。更有用的是：

- 当前任务在哪个阶段？
- 产物在哪里？
- 哪些内容已经审阅？
- 哪些数字、引用、图表或文件需要复查？
- 下一步是继续生成、人工决定、专业智能体修订，还是资源/权限需要处理？

这就是“结果带来路”。OPL App 让结果进入可复查、可解释、可继续的状态。它不把 evidence 当成给工程师看的日志，而是把 evidence 变成用户相信工作可以继续的原因。

7 工作台跟着工作走

OPL App 的产品定义不应停在“桌面应用”。更准确地说，它是本地优先、云端连续的专业工作台。

本地优先，意味着用户可以先在自己的 Mac 上打开 App，选择工作目录，围绕本机材料开始任务。敏感文件、个人项目、早期草稿、临时分析都可以先留在用户自己的工作环境里。

浏览器连续，意味着 Linux、Windows、服务器或云主机用户可以通过 Docker/WebUI 打开同一套 App 体验。对非 macOS 用户、远程机器、实验室服务器和容器部署来说，浏览器入口比桌面安装更自然。

云端连续，意味着当同一套 WebUI 被云端托管，并绑定账号、存储、隔离、资源策略和组织管理时，它就是 OPL Workspace。用户不需要从“本机 App”跳到“另一套云系统”。同一个项目、任务、产物、进度和回执语言继续存在，只是工作台运行的位置变了。

这条线可以概括为：

```
macOS OPL App
  -> Docker/WebUI on local/server/cloud VM
      -> hosted OPL Workspace
```

这样的设计让用户可以按需要逐步升级工作环境。今天在本机写基金，明天在服务器跑分析，后天在团队 Workspace 里审阅成果，用户面对的仍然是同一条工作线。

8 用已有资源，不重建世界

专业用户和团队通常已经有自己的文件、脚本、数据库、文献库、服务器、HPC、云主机、模型访问方式和内部工具。一个专业工作台不应该要求他们全部迁移到新世界。

OPL App 的方向是让已有资源进入同一条工作线：

计划 -> 批准 -> 执行 -> 监控 -> 回收 -> 回执

用户自己的本机文件可以作为材料进入任务。用户自带的 SSH、HPC 或云主机可以作为执行环境。OPL Gateway 可以作为 AI 能力入口。OPL Fabric 可以让计算、存储、环境、连接器和资源包进入统一资源语言。OPL Console 可以在组织需要时管理账号、权限、套餐、计费 and 资源策略。

重要的是，OPL App 不把这些资源变成一堆技术面板。用户应该先看到任务准备用什么资源、为什么用、是否需要批准、输出会在哪里、结束后回执是什么。

这样，资源能力服务于交付，而不是让用户从交付工作切换到资源管理工作。

9 为什么用户可以相信 OPL App 专业

专业感不是界面复杂，也不是内部名词多。用户愿意相信 OPL App，是因为它把复杂工作中最容易丢失的几件事保留下来。

第一，工作目的清楚。用户先选择科研、基金、演示、书稿或智能体构建，而不是选择后端。工作从目标出发，系统再组织 agent、工具和资源。

第二，进度能继续。App 不只显示一次回答。它让用户看到任务阶段、当前产物、卡点、下一步和可恢复上下文。

第三，结果有来路。产物能关联材料、代码、环境、审阅、回执和会话背景。几周后复查时，用户仍能理解结果从哪里来。

第四，专业边界清楚。App 展示专业智能体的工作线，但不替 MAS、MAG、RCA、BookForge 或 OMA 做领域质量裁决。研究质量、基金说服力、视觉交付质量、书稿完成度和智能体能力边界，仍回到对应专业 owner 和审阅环节。

第五，本地和云端连续。用户可以从自己的机器开始，也可以在浏览器、服务器或云端 Workspace 继续。工作台语言不变，用户不必为了切换运行位置而重新组织项目。

这些机制让 OPL App 不只是一个包装壳。它是把专业 AI 能力放进长期工作流程里的用户界面。

10 与 Framework、Cloud 和 Foundry Agents 的关系

OPL App 不是单独完成一切的产品。它处在 One Person Lab 的用户工作面位置。

OPL Framework 解决“复杂工作怎样可靠推进”。它负责阶段、工作空间、任务、运行、恢复、回执和状态投影。App 消费这些投影，让用户看到当前工作线。

OPL Cloud 解决“这些能力怎样在线化、资源化、组织化和可计量”。它包含 OPL Gateway、OPL Workspace、OPL Console、OPL Fabric、OPL Ledger 等能力。App 可以接入这些能力，但不把云端治理、计费或托管资源真相搬进本地界面。

Foundry Agents 解决“具体专业领域怎样判断质量并交付成果”。MAS、MAG、RCA、BookForge、OMA 和未来的 Patent、Award、Thesis、Review 等智能体，都保留自己的领域知识、检查标准、产物权威和交付边界。

OPL App 把三者连接成用户能使用的体验：

用户工作目的

- > OPL App 工作台
 - > OPL Framework 阶段运行
 - > Foundry Agent 专业工作
 - > OPL Cloud / Fabric / Gateway / Workspace 按需提供资源
 - > 产物、审阅、回执和下一步

这种关系让 One Person Lab 保持清楚：App 负责用户工作体验，Framework 负责运行骨架，Cloud 负责在线和资源能力，Foundry Agents 负责专业判断。

11 用户会如何感知 OPL App

用户不需要先理解 One Person Lab 的全部分层，才开始使用 OPL App。理想体验应该更直接。

第一次打开时，用户看到的是一个可以工作的入口：当前工作目录、可用模型、工作目的和输入框。App 应该让用户知道“现在能不能开始”，而不是让用户先读一组内部状态。

发起任务时，用户用普通语言描述目标。App 把目标放进合适的工作线：科研、基金、演示、书稿或智能体构建。用户不需要手动拼 agent、技能、工具和命令。

任务运行时，用户看到的是正在推进什么、最近产生了什么、是否需要自己批准资源或做决定。长任务不应变成黑盒，也不应把所有底层日志倒给用户。

任务结束时，用户得到的是产物和来路：文件在哪里，为什么生成，经过什么审阅，有哪些回执，下一轮可以从哪里继续。

换设备或换运行位置时，用户仍然沿用同一套语言。本机 App、浏览器 WebUI 和 OPL Workspace 的差异应该主要体现在访问方式、账号、存储和资源能力上，而不是让用户重新理解一套产品。

这就是 OPL App 白皮书要表达的专业性：不是把复杂系统讲复杂，而是把复杂系统变成用户能放心工作的界面。

12 本文边界

本文解释 OPL App 的产品理念和用户价值。它不是安装说明、设置说明、发布证明、运行状态证明或领域质量证明。

安装、更新、Docker/WebUI、Homebrew、Full first-install 和发布门由 App release 文档、contracts、release artifacts、CI workflow、VM smoke 和 owner receipts 维护。

运行状态、任务 projection、action receipt、provider diagnostics 和 stage ledger 由 OPL Framework 的 CLI/API、contracts、runtime readback 和 evidence surfaces 维护。

科研、基金、演示、书稿和智能体构建的质量判断由对应 Foundry Agent 和人工 owner 持有。OPL App 可以展示状态、产物和回执，但不把界面渲染包装成领域 ready、publication ready、funding ready、export ready 或 production

ready。

13 结语

OPL App 的目标不是让 AI 多回答几个问题，而是让一个人拥有类似专业团队的工作界面。

它让用户从一个工作台进入复杂知识工作，不用在工具之间来回跳。它让结果带来路，不只留下最终文件。它让工作台跟着工作走，从本机 App 到浏览器 WebUI，再到云端 OPL Workspace。它让专业智能体协作，但用户先看到的是自己的工作目的、当前产物和下一步。

这就是 OPL App 的位置：把 One Person Lab 的运行能力、专业智能体和云端资源，收敛成一个可信、可继续、可复查的专业工作台。